DOKUMENTASI PERANGKAT LUNAK

GUI Panel Kontrol Robot Feeder Akuatik

Pengembangan dengan Metode SDLC — Prototyping Model

Judul Skripsi	Pengembangan Robot Akuatik untuk Feeder Otomatis di Tambak
Sub-sistem	Transmitter — GUI Kendali (Raspberry Pi + NRF24L01)
Metode SDLC	Prototyping Model
Framework GUI	Python 3 + CustomTkinter
Tanggal	24 July 2026

1. Pendahuluan

1.1 Latar Belakang

Robot akuatik feeder otomatis telah berhasil beroperasi di air: mampu bergerak lurus dengan *heading-hold* berbasis MPU6050 dan berbelok melalui *differential thrust* dua motor brushless. Kendali sebelumnya dilakukan lewat antarmuka baris perintah (CLI) pada berkas **receiverArduino.py** yang mengharuskan operator mengetik dimensi tambak secara manual dan tidak menampilkan status sistem. Untuk meningkatkan kegunaan (*usability*) dan menyediakan pemantauan, dikembangkan antarmuka grafis (GUI) yang berjalan di Raspberry Pi.

1.2 Tujuan

- Menyediakan input dimensi tambak (panjang & lebar dalam meter) yang mudah.
- Menampilkan status koneksi modul NRF24 dan keterjangkauan robot secara nyata.
- Memberi kontrol manual untuk mematikan komponen yang relevan.
- Menampilkan telemetri (estimasi) kecepatan & RPM baling-baling secara realtime.
- Menampilkan data sensor MPU6050 (roll, pitch, yaw/heading) secara nyata via serial USB.
- Mencatat seluruh aktivitas dan mengekspornya sebagai berkas log (CSV/TXT).

1.3 Ruang Lingkup & Batasan

GUI berjalan di sisi **Transmitter** (Raspberry Pi) sebagai pengganti CLI. Firmware robot (ESP32, **Finalnew.ino**) tidak diubah pada iterasi ini. Konsekuensi batasan teknis yang teridentifikasi pada tahap analisis:

- **Komunikasi satu arah.** Raspberry hanya mengirim; robot hanya menerima. Robot tidak mengirim telemetri balik.
- **Tidak ada sensor RPM/encoder** pada robot. Maka nilai RPM & kecepatan yang ditampilkan adalah **ESTIMASI** yang dihitung dari nilai PWM throttle, bukan hasil pengukuran.
- **Kontrol komponen individual** (mis. mematikan satu motor atau MPU) memerlukan protokol dua arah — di luar lingkup iterasi ini; ditandai sebagai indikator *monitor*.

■ Kejujuran teknis ini justru menjadi bahan analisis yang baik: keterbatasan menjadi 'kebutuhan iterasi berikutnya' — sesuai semangat Prototyping.

2. Metodologi: SDLC Prototyping Model

Dipilih **Prototyping Model** karena kebutuhan GUI belum sepenuhnya pasti di awal dan paling baik ditemukan melalui purwarupa yang dievaluasi bersama dosen pembimbing. Cocok untuk sistem gabungan perangkat keras + lunak yang berevolusi.

2.1 Alasan Pemilihan (dibanding metode lain)

Metode	Kesesuaian untuk proyek ini
Waterfall	Rapi didokumentasikan, tetapi kaku bila kebutuhan berubah.
V-Model	Kuat di pengujian, tetapi mengasumsikan kebutuhan sudah pasti.
Incremental	Baik, mirip Prototyping namun tiap increment harus 'final'.

Metode	Kesesuaian untuk proyek ini
Prototyping (dipilih)	**Kebutuhan GUI ditemukan lewat purwarupa & umpan balik; iteratif; cocok HW+SW.**

2.2 Tahapan Prototyping

Fase	Aktivitas pada proyek ini
1. Pengumpulan Kebutuhan	Wawancara kebutuhan operator: input P×L, status NRF, kontrol komponen, telemetri realtime, ekspor log.
2. Perancangan Cepat (Quick Design)	Rancang tata letak GUI & pemisahan arsitektur core/GUI; definisikan protokol paket (misi/PING/STOP).
3. Bangun Prototipe	Implementasi feeder_core.py (RadioLink, MissionEstimator, ActivityLogger) dan feeder_gui.py (CustomTkinter).
4. Evaluasi oleh Pengguna	Demo ke pembimbing memakai MODE SIMULASI (tanpa hardware), lalu uji nyata di Raspberry Pi + robot.
5. Perbaikan (Refinement)	Revisi berdasar umpan balik (mis. label 'estimasi', LED status, penanganan input). Ulangi fase 3–5.
6. Rekayasa Produk & Pemeliharaan	Stabilkan, dokumentasikan, siapkan iterasi lanjutan (telemetri 2 arah).

2.3 Siklus Iterasi (Prototyping Loop)

```
Kebutuhan -> Quick Design -> Bangun Prototipe
^ |
| v
Perbaikan <---- Evaluasi Pengguna <---+
(ulangi hingga pengguna puas -> Produk Final)
```

3. Analisis Kebutuhan

3.1 Kebutuhan Fungsional

Kode	Kebutuhan	Status
RF-1	Input panjang & lebar tambak (meter) dan kirim misi AUTO	Terpenuhi
RF-2	Menampilkan status modul NRF & keterjangkauan robot (ACK)	Terpenuhi (real)
RF-3	Mematikan komponen yang dapat dikontrol (NRF TX, PING, Motor)	Terpenuhi
RF-4	Menampilkan RPM & kecepatan baling-baling realtime	Terpenuhi (estimasi)
RF-5	Mencatat & mengekspor log aktivitas (CSV/TXT)	Terpenuhi
RF-6	Perintah STOP membatalkan misi berjalan	Terpenuhi (real)
RF-7	Menampilkan data MPU6050 (roll/pitch/yaw) via serial USB	Terpenuhi (real)

3.2 Kebutuhan Non-Fungsional

- **Keterandalan:** heartbeat PING tiap 0,5 s menjaga failsafe robot (stop < 3 s bila sinyal hilang).
- **Portabilitas:** berjalan di Raspberry Pi (NRF real) & PC (auto-fallback simulasi).

- **Usability:** indikator warna (hijau/oranye/merah), label satuan jelas.
- **Kejujuran data:** nilai estimasi diberi label eksplisit agar tidak menyesatkan.

4. Perancangan (Quick Design)

4.1 Arsitektur Perangkat Lunak

Menerapkan *Separation of Concerns*: lapisan presentasi (GUI) dipisah dari logika (core), sehingga mudah diuji dan dikembangkan.

Modul	Tanggung jawab
feeder_gui.py	Presentasi: input, tombol, gauge, LED, tampilan log.
feeder_core.py → RadioLink	Abstraksi NRF24 (pyrf24) + auto-fallback MockRadio (simulasi).
feeder_core.py → MissionEstimator	Cermin timeline zigzag firmware → estimasi fase/PWM/RPM per waktu.
feeder_core.py → ActivityLogger	Pencatatan aktivitas berstempel waktu + ekspor CSV/TXT.

4.2 Protokol Komunikasi (tetap kompatibel dgn firmware)

Paket	Arah	Makna
P=<m>;L=<m>;MODE=AUTO	Pi → Robot	Perintah misi zigzag
PING	Pi → Robot	Heartbeat (jaga failsafe) tiap 0,5 s
STOP	Pi → Robot	Batalkan misi / matikan motor
(ACK otomatis)	Robot → Pi	Auto-ACK NRF; True = robot hidup & terjangkau

4.3 Dasar Estimasi Telemetri

Karena tak ada sensor, GUI menghitung PWM yang *seharusnya* dijalankan robot pada detik ke-t (mengikuti pola gerak firmware), lalu mengonversinya:

```
throttle = (PWM - 1000) / (2000 - 1000)
RPM_est = throttle * MOTOR_KV * BATTERY_V (default KV=2200, V=11.1)
speed_est= ROBOT_SPEED_MPS * throttle / throttle(CRUISE_PWM)
```

■ Konstanta MOTOR_KV, BATTERY_V, CRUISE_PWM, TURN_90_MS wajib disamakan dengan firmware & dikalibrasi pada pengujian air agar estimasi mendekati kenyataan.

4.4 Integrasi Data MPU6050 (Serial USB)

Berbeda dengan telemetri gerak yang berupa estimasi, data orientasi diambil **langsung dari sensor MPU6050**. Sketsa **mpu.ino** (Test Part/MPU Test) pada ESP32 mencetak orientasi ke Serial tiap 200 ms; GUI membacanya melalui **pyserial** pada jalur USB yang **terpisah dari NRF**. Kelas **MPUSerialReader** mem-parse baris berikut:

```
Angle -> X: <roll> Y: <pitch> Z(Yaw): <yaw> | Gyro -> X: <gx> Y: <gy> Z: <gz>
```

Roll (X), Pitch (Y), dan Yaw/heading (Z) ditampilkan realtime dan dapat direkam ke log (1 Hz) untuk diekspor. Jalur serial cocok untuk fase **kalibrasi/uji bench** saat board tertambat kabel; saat robot

berlayar bebas, data MPU realtime memerlukan telemetri NRF dua arah (iterasi berikutnya).

5. Implementasi Prototipe

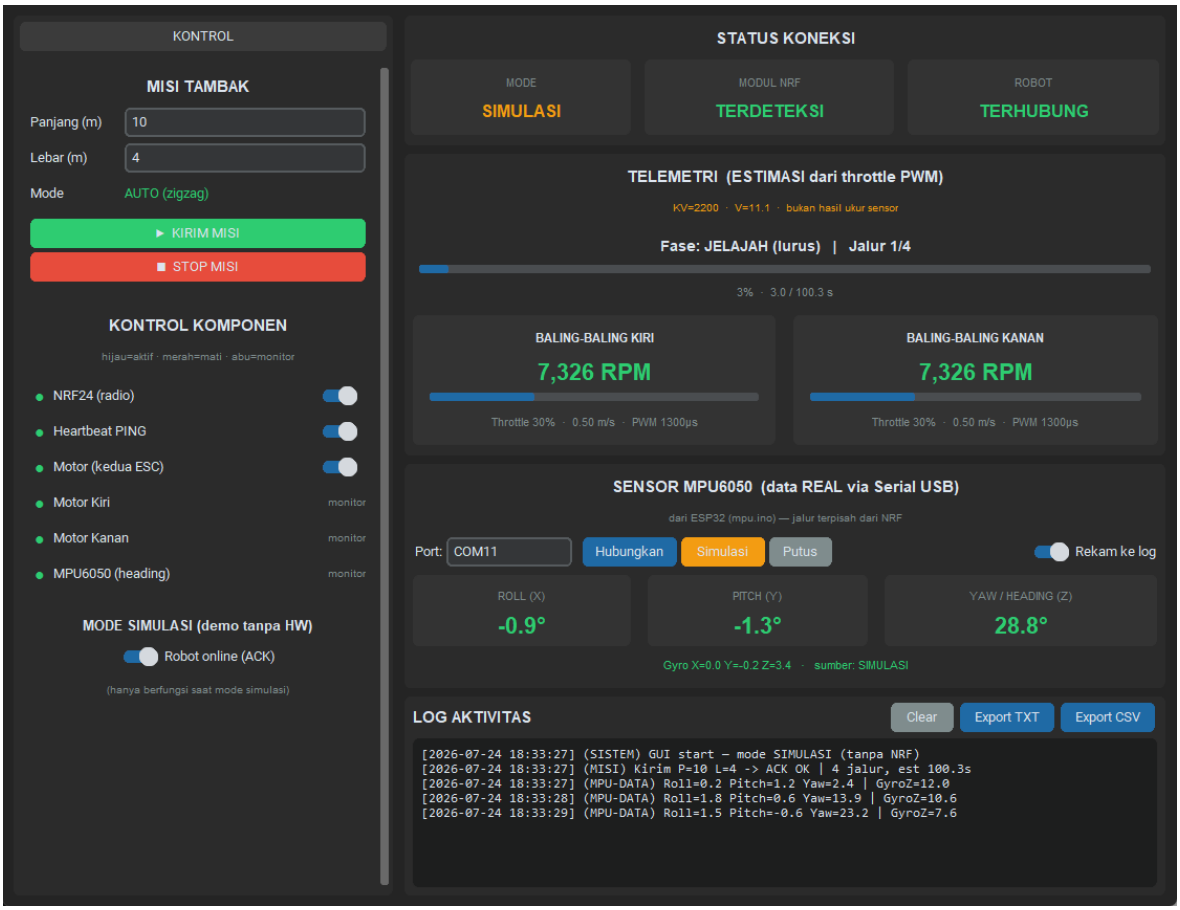
5.1 Fitur GUI

- **Panel Misi:** input Panjang & Lebar (m), tombol KIRIM MISI & STOP.
- **Status Koneksi:** mode (REAL/SIMULASI), deteksi modul NRF, status robot.
- **Kontrol Komponen:** saklar NRF TX, Heartbeat PING, Motor (efek nyata lewat STOP); indikator monitor Motor Kiri/Kanan & MPU.
- **Telemetri:** fase misi, progress bar, gauge RPM/throttle/speed L & R.
- **Sensor MPU6050:** pilih port serial, tampil Roll/Pitch/Yaw realtime, opsi rekam ke log.
- **Log:** tampilan realtime + tombol Export CSV / Export TXT / Clear.

5.2 Mode Simulasi

Bila pyrf24 tidak tersedia atau NRF tak terdeteksi, RadioLink otomatis memakai MockRadio; demikian pula MPUSerialReader menyediakan mode simulasi bila serial tidak tersedia. Dengan begitu GUI tetap dapat didemokan dan diuji tanpa perangkat keras — sangat membantu pada fase evaluasi Prototyping.

5.3 Tampilan Antarmuka



Gambar 1. Antarmuka GUI saat misi 10×4 m berjalan (jalur 1/4), gauge RPM & panel MPU6050 aktif, log aktivitas terisi.

6. Pengujian (Black-Box)

Uji	Skenario	Hasil Diharapkan
T-1	Input P=10 L=4, klik KIRIM MISI	Paket terkirim, 4 jalur terhitung, gauge bergerak
T-2	Robot online (ACK) di simulasi	Status ROBOT = TERHUBUNG (hijau)
T-3	Robot offline	Status ROBOT = TERPUTUS (merah) < 3 s
T-4	Matikan saklar Motor	STOP terkirim, misi berhenti, LED motor merah
T-5	Input non-angka	Muncul dialog error, misi tidak dikirim
T-6	Klik Export CSV/TXT	Berkas log tersimpan & berisi seluruh event
T-7	Timeline estimasi habis	Fase = SELESAI, misi & heartbeat berhenti otomatis
T-8	Hubungkan MPU (serial/simulasi) + Rekam ke log	Roll/Pitch/Yaw tampil realtime, data MPU masuk log & ekspor

7. Panduan Instalasi & Penggunaan

7.1 Instalasi

```
# Di Raspberry Pi (kendali nyata):
pip install customtkinter pyrf24

# Di PC (mode simulasi, tanpa NRF):
pip install customtkinter
```

7.2 Menjalankan

Cukup jalankan satu berkas entry point **receiverArduino.py**; GUI, telemetri, dan logging berjalan otomatis:

```
cd Master Code/Transmitter
python receiverArduino.py # buka GUI (default)
python receiverArduino.py --cli # mode teks lama (opsional)
```

7.3 Berkas Terkait

Berkas	Keterangan
receiverArduino.py	ENTRY POINT — jalankan ini → GUI (opsi --cli utk mode teks lama)
feeder_gui.py	Aplikasi GUI (CustomTkinter)
feeder_core.py	Backend: RadioLink, MissionEstimator, ActivityLogger, MPUSerialReader
capture_gui.py	Skrip pengambil screenshot GUI untuk dokumentasi
build_docs_pdf.py	Generator dokumen PDF ini
Robot/New/Finalnew/Finalnew.ino	Firmware robot (ESP32) — tidak diubah
Test Part/MPU Test/mpu/mpu.ino	Sketsa sumber data MPU6050 via serial

7.4 Deploy di Raspberry Pi (rekomendasi)

GUI dijalankan di Raspberry Pi karena memiliki GPIO/SPI untuk modul NRF24. GUI membutuhkan sesi **Desktop** (monitor HDMI atau VNC) — tidak bisa lewat SSH polos. Wiring NRF24L01 mengikuti pemetaan pin berikut (kode: RF24(24, 0) → CE=GPIO24, CSN=CE0):

NRF24L01	Raspberry Pi (BCM)	Pin fisik
VCC (3.3V)	3.3V	1 / 17
GND	GND	6, 9, ...
CE	GPIO24	18
CSN	GPIO8 (CE0)	24
SCK	GPIO11	23
MOSI	GPIO10	19
MISO	GPIO9	21

■ VCC NRF wajib 3.3V (bukan 5V); tambahkan kapasitor 10µF antara VCC–GND modul agar catu daya stabil.

Instalasi otomatis — ekstrak paket **raspberry_feeder.zip**, lalu jalankan skrip setup sekali:

```
cd ~/feeder # lokasi hasil ekstrak
chmod +x setup_pi.sh run.sh
./setup_pi.sh # aktifkan SPI, apt, venv, pip install
```

Instalasi manual (bila ingin bertahap):

```
sudo raspi-config nonint do_spi 0 # aktifkan SPI
sudo apt install -y python3-venv python3-tk
sudo usermod -aG spi,gpio,dialout $USER # lalu logout-login
python3 -m venv --system-site-packages ~/feeder-venv
source ~/feeder-venv/bin/activate
pip install -r requirements.txt
```

Menjalankan:

```
./run.sh # buka GUI
./run.sh --cli # mode teks lama
```

■ Gunakan venv dengan --system-site-packages agar python3-tk (dari apt) terlihat di dalam venv; tanpa itu GUI gagal import tkinter. Bila status MODE tetap SIMULASI di Pi, berarti SPI belum aktif / wiring NRF salah (cek: ls /dev/spidev*).

8. Kesimpulan & Iterasi Berikutnya

Prototipe GUI berhasil memenuhi seluruh kebutuhan fungsional sesuai lingkup iterasi ini, dengan pemisahan arsitektur yang bersih dan mode simulasi yang mempermudah evaluasi. Sesuai Prototyping Model, keterbatasan yang teridentifikasi menjadi rencana iterasi selanjutnya:

- Menambah **telemetri dua arah** (ACK payload NRF) agar status komponen & throttle asli dikirim balik dari robot.
- Memasang **sensor RPM (hall/encoder)** untuk mengganti estimasi dengan data ukur.
- Menambah perintah firmware untuk **kontrol komponen individual** (motor kiri/kanan, MPU).

- Kalibrasi konstanta (ROBOT_SPEED_MPS, TURN_90_MS, KV) pada pengujian air.
